

Name: Oraye, Louise Justine J.

Subject: 6ADET

Block Section: CS - 301

Date: September 18, 2026

COOKIE BITES

The problem, in one sentence

I currently spend a lot of time manually recording customer orders, payments, and profits in spreadsheets, making it easy to miss updates or make calculation errors, especially when handling multiple orders.

Who is this for?

This app is designed for sweet and pastry business owners, such as students or home-based entrepreneurs who sell through online like me.

Currently, I manage orders using Google Sheets, notebooks, or manual calculations, then compute sales, costs, and profits separately.

Core Features (MVP)

#	Feature	Still in the MVP? keep / cut / stretch	Flutter pieces it needs	Honest Estimate
1	Login Page	Keep	● LoginPage ● TextFormField for username/password ● ElevatedButton ● Form validation ● Navigator to Order Board ● Simple authentication/storage	5-7 Hours
2	Order Board	Keep	● OrderBoardPage ● ListView.builder ● OrderCard reusable component ● Filter using DropDownButton/ChoiceChip ● Edit/update functions ● Payment status control	7-10 Hours

3	Order Entry Form	Keep	● OrderFormPage ● TextFormField ● DropDownButtonFormField ● Quantity input ● Payment selection ● Order model class ● Calculation functions ● ElevatedButton to save order	7-10 Hours
4	Expense Log	Keep	● ExpenseLogPage ● TextFormField ● Expense category dropdown ● Expense model class ● ListView.builder ● Add/Edit/Delete functions total expense calculation	7-10 Hours
5	Add Expense (Pops out on Expense Log only, doesn't have own page)	Keep	● showDialog() ● AlertDialog/custom Dialog ● Form ● TextFormField ● DropDownButtonFormField ● DatePicker ● Validation ● Add Entry button ● Expense model	5-7 Hours
6	Financial Summary Dashboard	Keep	● DashboardPage ● Reusable summary cards ● Calculation functions ● Column/Row or GridView ● Card widgets ● Data from orders and expenses	5-7 Hours

Total Estimate: 36–51 hours total across all six features.

Time Availability Check: With approximately 3 weeks remaining in the term and an average of 10–12 dedicated development hours per week (totaling around 40–45 available hours), this 36–51 hour estimate fits tightly within the remaining schedule.

Stretch goals

Move everything you cut into this list. It should be growing, not shrinking, and that is a sign of good planning, not of giving up.

1. Landing Page (Merged with Login page)
2. Menu Page (Merged with Order Board through Navigation Buttons)

NEW: How my app saves data

1. **The question from page 1: if two different people install my app, should they see the same data?**

No, this app is a private productivity and financial tracking tool for individual home-based pastry and sweet entrepreneurs. Each user handles their own independent customers, orders, ingredient costs, and business profits, meaning data must stay completely private and separate between different users.

2. **Roughly how many records does my app hold in a realistic week of use?**

30 to 60 records per week. This includes approximately 15 to 30 customer order entries (tracked via the Order Board) and 10 to 20 raw material or utility expense logs.

3. **My choice: name it.**

Supabase (Cloud-hosted PostgreSQL database with built-in user authentication)

4. **Why this one and not the others, in two or three sentences. Name the tradeoff you accepted.**

Supabase is chosen because it allows business owners to create accounts and securely log in so their orders and financial data are tied to their profile and accessible across devices. The tradeoff accepted is that the app now requires an active internet connection to authenticate users and sync data, adding setup complexity compared to a local-only database.

5. What I save, concretely: which class, which fields, under what key or in which table or collection.

Database Table	Primary Key / ID	Foreign Key / Account Link	Fields Saved
orders	id (UUID)	user_id (UUID linked to Auth)	● customer_name (text) ● item_ordered (text) ● quantity (int) ● total_price (numeric) ● payment_status (text) ● order_date (timestamp) ● fulfillment_type (text) ● flavor (text) ● payment mode (text) ● adjustment_note (text)
expenses	id (UUID)	user_id (UUID linked to Auth)	● category (text) ● amount (numeric) ● date (timestamp) ● expense_name (text) ● description (text)

6. Have I tried it yet? If you did the one-hour spike from page 6, say what happened. If not, say when you will.

No, but I will run the one-hour spike once I submit this proposal to set up the Supabase project, configure user authentication, and test inserting a record into the database tables.

NEW: One thing I want to add that the course did not teach

Nothing, I am still figuring Flutter out and want to focus on nailing the core basics first. Because I am concentrating entirely on making my essential MVP features to work properly, I don't have

an idea yet for something outside the course material, and I want to make sure my main app is fully complete and functional before trying to add advanced extras.

NEW: How my project runs when someone else opens it

I am keeping the device_preview wrapper:

No, because it can sometimes interfere with testing responsiveness or standard layout behavior when evaluators check the app directly in a browser.

My app runs in a browser with flutter run -d web-server, start to finish, with every screen reachable:

Not yet, because I am still finishing up the wiring for the navigation routes between the order board, expense log, and financial dashboard.

Anything that needs real hardware degrades to sample data instead of crashing:

Not applicable, since my app relies on Supabase cloud database and form inputs rather than device-specific hardware like cameras or Bluetooth sensors.

Does anything I plan to build need an API key, a password, or real personal data?

Yes, my app uses Supabase API keys and a project URL to connect to the cloud database, which will be safely stored in a git-ignored .env file locally and via repository secrets for deployment, while user accounts will handle passwords securely through Supabase Auth without hardcoding any real personal data into the repo.

Data the App Needs to Remember

Thing	Fields	Where it is saved
Orders	• Customer Name • Delivery Date • Fulfillment Type (Pick-up, HAU, Delivery) • Flavor • Quantity • Amount • COGS • Profit • Payment Mode • Adjustment/Short Note	Supabase orders table
Expenses	• Expense Name	Supabase expenses table

	• Date • Description • Category • Amount	
--	---	--

Screens

1. Login Page
2. Order Board
3. Order Entry Form
4. Expense Log
5. Add Expense (Pops out on Expense Log Screen Only)
6. Financial Summary Dashboard

One Risk

1. **The risk I named last time:** Implementing automatic pricing and financial calculations instantly whenever orders or expenses change. Building M4 and M5 made this risk smaller because I now have hands-on experience handling form states and ListView builders, but coordinating updates across multiple screens remains challenging.
 - **First step:** Prototype the profit-calculation function in isolation.
2. **A new risk I did not see before:** Ensuring seamless data synchronization across the Order Board, Expense Log, and Financial Summary Dashboard so that records modified in one view immediately update totals elsewhere without state drift.
 - **First step:** Build a centralized state management utility or provider callback linking order actions to the dashboard metrics.

What changed, and why

Section	Prelim said	Now says	Why it changed?
Core Features (MVP)	4 core features (excluding standalone landing page, login, and menu pages)	6 features tracked in the MVP table (including collapsed login/landing page flow, and the add-expense pop-up)	I collapsed the landing and login pages into one entry flow, integrated the menu page into the order board via navigation, and explicitly added the "Add Expense" dialog to keep my development tracking organized.

Screens	1. Landing Page 2. Login Page 3. Menu Page 4. Master Order Board 5. Order Entry Form 6. Expense Log 7. Add Expense (Pops out on Expense Log Screen) 8. Financial Summary Dashboard	1. Login Page 2. Master Order Board 3. Order Entry Form 4. Expense Log 5. Add Expense (Pops out on Expense Log Screen Only) 6. Financial Summary Dashboard	Updated to match the collapsed login/landing flow and ensure every functional view and modal dialog has a clear home in the wireframes
---------	---	---	--